

Dependencies in Formal Mathematics: Applications and Extraction for Coq and Mizar

Jesse Alama^{1,*}, Lionel Mamane^{2,**}, and Josef Urban^{3,***}

¹ Center for Artificial Intelligence
New University of Lisbon

j.alama@fct.unl.pt
<http://centria.di.fct.unl.pt/~alama/>

² 59, rue du X Octobre
L-7243 Bereldange
Luxembourg
lionel@mamane.lu

³ Institute for Computing and Information Sciences
Radboud University Nijmegen
josef.urban@gmail.com

Abstract. Two methods for extracting detailed formal dependencies from the Coq and Mizar system are presented and compared. The methods are used for dependency extraction from two large mathematical repositories: the Coq Repository at Nijmegen and the Mizar Mathematical Library. Several applications of the detailed dependency analysis are described and proposed. Motivated by the different applications, we discuss the various kinds of dependencies that we are interested in, and the suitability of various dependency extraction methods.

1 Introduction

This paper presents two methods for extracting detailed formal dependencies from two state-of-the-art interactive theorem provers (ITPs) for mathematics: the Coq system and the Mizar system. Our motivation for dependency extraction is application-driven. We are interested in using detailed dependencies for fast refactoring of large mathematical libraries and wikis, for AI methods in automated reasoning that learn from previous proofs, for improved interactive editing of formal mathematics, and for foundational research over formal mathematical libraries.

* Supported by the ESF research project *Dialogical Foundations of Semantics* within the ESF Eurocores program *LogICCC* (funded by the Portuguese Science Foundation, FCT LogICCC/0001/2007). Research for this paper was partially done while a visiting fellow at the Isaac Newton Institute for the Mathematical Sciences in the program ‘Semantics & Syntax’.

** Supported during part of the research presented here by the NWO project “Formal Interactive Mathematical Document: Creation and Presentation”; during that time, he was affiliated with the ICIS, Radboud University Nijmegen.

*** Supported by the NWO project “MathWiki: A Web-based Collaborative Authoring Environment for Formal Proofs”.

These applications require different notions of *formal dependency*. We discuss these different requirements, and as a result provide implementations that in several important aspects significantly differ from previous methods. For Mizar, the developed method captures practically all dependencies needed for successful re-verification of a particular formal text (i.e., also notational dependencies, automations used, etc.), and the method attempts hard to determine the minimal set of such dependencies. For Coq, the method goes farther towards re-verification of formal texts than previous methods [5,13,4] that relied solely on the final proof terms. For example, we can already track Coq dependencies that appear during the tactic interpretation, but that do not end up being used in the final proof term.

The paper is organized as follows. Section 2 briefly discusses the notion of formal dependency. Section 3 describes the implementation of dependency extraction in the Coq system, and Section 4 describes the implementation in the Mizar system. Section 5 compares the two implemented approaches to dependency computation. Section 6 describes several experiments and measurements conducted using our implementations on the CoRN and MML libraries, including training of AI/ATP proof assistance systems on the data, and estimating the speed-up for collaborative large-library developments. Section 8 concludes.

2 Dependencies: What Depends on What?

Generally, we say that a definition, or a theorem, T *depends* on some definition, lemma or other theorem T' , (or equivalently, that T' is a *dependency* of T) if T “needs” T' to exist or hold. The main way such a “need” arises is that the well-formedness, justification, or provability of T does not hold in the absence of T' . We consider formal mathematics done in a concrete proof assistant so we consider mathematical and logical constructs not only as abstract entities depending on each other, but also as concrete objects (e.g., texts, syntax trees, etc.) in the proof assistants. For our applications, there are at least two different notions of “dependency” we are interested in:

- Semantic/logical: One might claim, for example, that in the Coq context a λ -term (or proof object in the underlying formal framework) contains all dependencies of interest for a particular theorem, regardless of any notational conventions, library mechanisms, etc.
- Pragmatic: Such dependencies are met if a particular item still compiles, regardless of possibly changed underlying semantics. This view takes the whole proof assistant as the locus of dependency, with its sophisticated mechanisms such as auto hint databases, notations, type automations, definitions expansions, proof search depth, parser settings, hidden arguments, etc.

Formal dependencies can also be implicit and explicit. In the simple world of first-order automated theorem proving, proofs and their dependencies are generally quite detailed and explicit about (essentially) all logical steps, even very small ones (such as the steps taken in a resolution proof). But in ITPs, which are

generally oriented toward human mathematicians, one of the goals is to allow the users to express themselves with minimal logical verbosity and ITPs come with a number of implicit mechanisms. Examples are type mechanisms (e.g., type-class automations of various flavors in *Coq* [14] and *Isabelle* [8], Prolog-like types in *Mizar* [17,15]), hint mechanisms (in *Coq* and *Isabelle*), etc. If we are interested in giving a complete answer to the question of what a formalized proof depends upon, we must expose such implicit facts and inferences.

Formal dependencies reported by ITPs are typically *sufficient*. Depending on the extraction mechanism, redundant dependencies can be reported. Bottom-up procedures like congruence-closure and type closure in *Mizar* (and e.g., type-class mechanisms in other ITPs) are examples of mechanisms when the ITP uses available knowledge exhaustively, often drawing in many *unnecessary* dependencies from the context. For applications, it is obviously better if such unnecessary dependencies can be removed.

3 Dependency Extraction in *Coq*

Recall that *Coq* is based on the Curry-Howard isomorphism, meaning that:

1. A statement (formula) is encoded as a type.
2. There is, at the “bare” logical level, no essential difference between a definition and a theorem: they are both the binding (in the environment) of a name to a type (type of the definition, statement of the theorem) and a term (body of the definition, proof of the theorem).
3. Similarly, there is no essential difference between an axiom and a parameter: they are both the binding (in the environment) of a name to a type (statement of the axiom, type of the parameter, e.g. “natural number”).
4. There is, as far as *Coq* is concerned, no difference between the notions of theorem, lemma, corollary, ...

The type theory implemented by *Coq* is called the predicative calculus of inductive constructions, abbreviated as pCIC.

There are essentially three groups of *Coq* commands that need to be treated by the dependency tracking:¹

1. Commands that register a new logical construct (definition or axiom), either
 - From scratch. That is, commands that take as arguments a name and a type and/or a body, and that add the definition binding this name to this type and/or body. The canonical examples are

Definition Name : type := body

and

Axiom Name : type

The type can also be given implicitly as the inferred type of the body, as in

¹ As far as logical constructs are concerned.

Definition Name := body

- Saving the current (completely proven) theorem in the environment. These are the “end of proof” commands, such as **Qed**, **Save**, **Defined**.
- 2. Commands that make progress in the current proof, which is necessarily made in several steps:
 - (a) Opening a new theorem, as in

Theorem Name : type

or

Definition Name : type

- (b) An arbitrary strictly positive amount of proof steps.
- (c) Saving that theorem in the environment. These commands update (by adding exactly *one* node) the internal **Coq** structure called “proof tree”.
3. Commands that open a new theorem, that will be proven in multiple steps.

The dependency tracking is implemented as suitable hooks in the **Coq** functions that the three kinds of commands eventually call. When a new construct is registered in the environment, the dependency tracking walks over the type and body (if present) of the new construct and collects all constructs that are referenced. When a proof tree is updated, the dependency tracking examines the top node of the new proof tree (note that this is always the only change with regards to the previous proof tree). The commands that update the proof tree (that is, make a step in the current proof) are called **tactics**. **Coq**’s tactic interpretation goes through three main phases:

1. parsing;
2. Ltac² expansion;
3. evaluation.

The tactic structure after each of these phases is stored in the proof tree. This allows to collect all construct references mentioned at any of these tree levels. For example, if tactic **Foo T** is defined as

```
try apply BolzanoWeierstrass;
solve [ T | auto ]
```

and the user invokes the tactic as **Foo FeitThompson**, then the first level will contain (in parsed form) **Foo FeitThompson**, the second level will contain (in parsed form)

```
try apply BolzanoWeierstrass;
solve [ FeitThompson | auto ].}
```

and the third level can contain any of:

² Ltac is the **Coq**’s tactical language, used to combine tactics and add new user-defined tactics.

- `refine (BolzanoWeierstrass ...)`,
- `refine (FeitThompson ...)`,
- something else, if the proof was found by `auto`.

The third level typically contains only a few of the basic atomic fundamental rules (tactics) applications, such as `refine`, `intro`, `rename` or `convert`, and combinations thereof.

3.1 Dependency Availability, Format, and Protocol

Coq supports several interaction protocols: the `coqtop`, `emacs` and `coq-interface` protocols. Dependency tracking is available in the program implementing the `coq-interface` protocol which is designed for machine interaction. The dependency information is printed in a special message for each *potentially progress-making command* that can give rise to a dependency.³ A *potentially progress-making command* is one whose purpose is to change Coq’s state. For example, the command `Print Foo`, which displays the previously loaded mathematical construct `Foo`, is not a potentially progress-making command⁴. Any tactic invocation is a potentially progress-making command. For example, the tactic `auto` silently succeeds (without any effect) if it does not completely solve the goal it is assigned to solve. In that case, although that particular invocation did not make any actual progress in the proof, `auto` is still considered a potentially progress-making command, and the dependency tracking outputs the message ‘‘dependencies: (empty list)’’. Other kinds of progress-making commands include, for example notation declarations and morphisms registrations. Some commands, although they change Coq’s state, might not give rise to a dependency. For example, the `Set FirstorderDepth` command, taking only an integer argument, changes the maximum depth at which the `firstorder` tactic will search for a proof. For such a command, no dependency message is output.

One command may give rise to several dependency messages, when they change Coq’s state in several different ways. For example, the `intuition` tactic⁵ can, mainly for efficiency reasons, construct an ad hoc lemma, register it into the global environment and then use that lemma to prove the goal it has been assigned to solve, instead of introducing the ad hoc lemma as a local hypothesis through a cut. This is mainly an optimization: The ad hoc lemma is defined as

³ In other words, the system gives dependencies of individual *proof steps*, not only of whole proofs.

⁴ Thus, although this commands obviously needs item `Foo` to be defined to succeed, the dependency tracking does not output that information. That is not a problem in practice because such commands are usually issued by a user interface to treat an interactive user request (e.g. “show me item `Foo`”), but are not saved into the script that is saved on disk. Even if they were saved into the script, adding or removing them to (from, respectively) the script does not change the semantics of the script.

⁵ The intuition tactic is a decision procedure for intuitionistic propositional calculus based on the contraction-free sequent calculi LJT* of Roy Dyckhof, extended to hand over subgoals which it cannot solve to another tactic.

“opaque”, meaning that the typechecking (proofchecking) algorithm is not allowed to unfold the body (proof) of the lemma when the lemma is invoked, and thus won’t spend any time doing so. By contrast, a local hypothesis is always “transparent”, and the typechecking algorithm is allowed to unfold its body. For the purpose of dependency tracking this means that `intuition` makes *two* conceptually different steps:

1. register a new global lemma, under a fresh name;
2. solve the current subgoal in the proof currently in progress.

Each of these steps gives rise to different dependencies. For example, if the current proof is `BolzanoWeierstrass`, then the new global lemma gives rise to dependencies of the form

“`BolzanoWeierstrass_subproofN` depends on ...”

where the `_subproofN` suffix is Coq’s way of generating a fresh name. Closing of the subgoal by use of `BolzanoWeierstrass_subproofN` then gives rise to the dependency

“`BolzanoWeierstrass` depends on `BolzanoWeierstrass_subproofN`”

3.2 Coverage and Limitations

The Coq dependency tracking is already quite extensive, and sufficient for the whole Nijmegen CoRN corpus. Some restrictions remain in parts of the Coq internals that the second author does not yet fully understand.⁶ Our interests (and experiments) include not only purely mathematical dependencies that can be found in the proof terms (for previous work see also [13,4]), but also fast recompilation modes for easy authoring of formal mathematics in large libraries and formal wikis. The Coq dependency tracking code currently finds all logically relevant dependencies from the proof terms, even those that arise from automation tactics. It does not handle yet the non-logical dependencies. Examples include notation declarations, morphism and equivalence relation declarations,⁷ `auto` hint database registrations,⁸ but also tactic interpretation. At this stage, we don’t handle most of these, but as already explained, the internal structure of Coq lends itself well to collecting dependencies that appear at the various levels of tactic interpretation. This means that we can already handle the (*non-semantic*) dependencies on logical constructs that appear during the tactic interpretation, but that do not end up being used in the final proof term.

⁶ E.g. a complete overview of all datatypes of tactic arguments and how “dynamics” are used in tactic expressions.

⁷ So that the tactics for equality can handle one’s user-defined equality.

⁸ `auto` not only needs that the necessary lemmas be available in the environment, but it also needs to be specifically instructed to try to use them, through a mechanism where the lemmas are registered in a “hint database”. Each invocation of `auto` can specify which hint databases to use.

Some of the non-logical dependencies are a more difficult issue in practice, albeit not always in theory. For example, several dependencies related to tactic parametrization (`auto` hint databases, `firstorder` proof depth search, morphism declarations) need specific knowledge of how the tactic is influenced by parameters or previous non-logical declarations. The best approach to handle such dependencies seems to be to change (at the OCaml source level in Coq) the type of a tactic, so that the tactic itself is responsible for providing such dependencies. This will however have to be validated in practice, provided that we manage to persuade the greater Coq community about the importance and practical usefulness of complete dependency tracking for formal mathematics and for research based on it.

Coq also presents an interesting corner case as far as opacity of dependencies is concerned. On the one hand, Coq has an explicit management of opacity of items; an item originally declared as opaque can only be used generically with regards to its type; no information arising from its body can be used, the only information available to other items is the type. Lemmas and theorems are usually declared opaque⁹, and definitions usually declared transparent, but this is not forced by the system. In some cases, applications of lemmas need to be transparent. Coq provides an easy way to decide whether a dependency is opaque or transparent: dependencies on opaque objects can only be opaque, and dependencies on transparent objects are to be considered transparent.

Note that the pCIC uses a universe level structure, where the universes have to be ordered in a well-founded way at all times. However, the ordering constraints between the universes are hidden from the user, and are absent from the types (statements) the user writes. Changing the proof of a theorem T can potentially have an influence on the universe constraints of the theorem. Thus, changing the body of an opaque item T' appearing in the proof of T can change the universe constraints attached to it, potentially in a way that is incompatible with the way it was previously used in the body of T . Detecting whether the universe constraints have changed or not is not completely straightforward, and needs specific knowledge of the pCIC. But unless one does so, for complete certainty of correctness of the library as a whole, one has to consider *all* dependencies as transparent. Note that in practice universe constraint incompatibilities are quite rare. A large library may thus optimize its rechecking after a small change, and not immediately follow opaque reverse dependencies. Instead, fully correct universe constraint checking could be done in a postponed way, for example by rechecking the whole library from scratch once per week or per month.

4 Dependency Extraction in Mizar

Dependency computation in Mizar differs from the implementation provided for Coq, being in some sense much simpler, but at the same time also more robust with respect to the potential future changes of the Mizar codebase. For comparison of the techniques, see Section 5. For a more detailed discussion of Mizar, see [11] or [7].

⁹ Thereby following the mathematical principle of **proof irrelevance**.

In Mizar, every article A has its own environment $\mathcal{E}_A$ specifying the context (theorems, definitions, notations, etc.) that is used to verify the article. $\mathcal{E}_A$, is usually a rather conservative overestimate of the items that the article actually needs. For example, even if an article A needs only one definition (or theorem, or notation, or scheme, or . . .) from article B , all the definitions (theorems, notations, schemes, . . .) from B will be present in $\mathcal{E}_A$. The *dependencies for an article* A are computed as the smallest environment $\mathcal{E}'_A$ under which A is still Mizar-verifiable (and has the same semantics as A did under $\mathcal{E}_A$). To get dependencies of a particular Mizar item I (theorem, definition, etc.), we first create a *microarticle* containing essentially just the item I , and compute the dependencies of this microarticle.

More precisely, computing fine-grained dependencies in Mizar takes three steps:

Normalization. Rewrite every article of the Mizar Mathematical Library so that:

- Each definition block defines exactly one concept.
(Definition blocks that contain multiple definitions or notations can lead to false positive dependencies.)
- All toplevel logical linking is replaced by explicit reference: constructions such as

ϕ ; **then** ψ ;

whereby the statement ψ is justified by the statement ϕ , are replaced by

Label1: ϕ ;
 Label2: ψ by Label1;

where **Label1** and **Label2** are new labels. By doing this transformation, we ensure that the only way that a statement is justified by another is through explicit reference.

- Segments of reserved variables all have length exactly 1. For example, constructions such as

reserve A **for** set ,
 B **for** non empty set ,
 f **for** Function **of** A, B,
 M **for** Cardinal;

which is a single reservation statement that assigns types to four variables (A, B, f, and M) is replaced by four reservation statements, each of which assigns a type to a single variable:

reserve A **for** set;
reserve B **for** non empty set;
reserve f **for** Function **of** A, B;
reserve M **for** Cardinal;

When reserved variables are normalized in this way, one can eliminate some false positive dependencies. A theorem in which, say, the variable **f** occurs freely but which has nothing to do with cardinal numbers has the type **Function of A,B** in the presence of both the first and the

second sequences of reserved variables. If the first reservation statement is deleted, the theorem becomes ill-formed because $\mathbf{f}$ no longer has a type. But the reservation statement itself directly requires that the type **Cardinal** of cardinal numbers is available, and thus indirectly requires a part of the development of cardinal numbers. If the theorem has nothing to do with cardinal numbers, this dependency is clearly specious.

These rewritings do not affect the semantics of the **Mizar** article.

Decomposition. For every normalized article A in the **Mizar** Mathematical Library, extract the sequence $\langle I_1^A, I_2^A, \dots, I_n^A \rangle$ of its toplevel items, each of which written to a “microarticle” A_k that contains only I_k^A and whose environment is that of A and contains each A_j ($j < k$).

Minimization. For every article A of the **Mizar** Mathematical Library and every microarticle A_n of A , do a brute-force minimization of smallest environment $\mathcal{E}_{A_n}$ such that A_n is **Mizar**-verifiable.

The brute-force minimization works as follows. Given a microarticle A , we successively trim the environment.¹⁰ For each item kind we have a sequence s of imported items $\langle a_1, \dots, a_n \rangle$, from which we find a minimal sublist s' with respect to which A is **Mizar**-verifiable.¹¹ Applying this approach for all **Mizar** item kinds, for all microarticles A_k , for all articles A of the MML is a rather expensive computation (for some **Mizar** articles, this process can take several hours). It is much slower than the method used for **Coq** described in Section 3. However the result is truly minimized, which is important for many applications of dependencies. Additionally, the minimization can be made significantly faster by applying heuristics, see Section 6.4 for a learning-assisted approach.

5 Comparison of the Methods

Some observations comparing the **Coq** and **Mizar** dependency computation can be drawn generally, without comparing the actual data as done in the following sections.

Dependencies in the case of **CoRN** are generated by hooking into the actual code and are thus quite exactly mirroring the work of the proof assistant. In the case of **Mizar**, dependencies are approximated from above. The dependency graph in this case starts with an over-approximation of what is known to be sufficient for an item to be **Mizar**-verifiable and then successively refines this over-approximation toward a minimal set of sufficient conditions. A significant difference is that the dependencies in **Coq** are not minimized. The dependency tracking there tells us exactly the dependencies that were used by **Coq** (in the particular context) when a certain command is run. For example, if the context is rich, and redundant dependencies are used by some exhaustive strategies, we will not detect their redundancy. On the other hand, in **Mizar** we do not rely

¹⁰ Namely, theorems, schemes, top-level lemmas, definitional theorems, definientia, patterns, registrations, and constructors. See [7] for a discussion of these item kinds.

¹¹ There is a (possibly non-unique) minimal sublist, since we assume that A is **Mizar**-verifiable to begin with.

on its exact operation. There, we exhaustively minimize the set of dependencies until an error occurs. This process is more computationally intensive. However, it does guarantee minimality (relative to the proof assistant’s power) which is interesting for many of the applications mentioned below.

Another difference is in the coverage of non-logical constructs. Practically every resource necessary for a verification of a Mizar article is an explicit part of the article’s environment. Thus, it is easy to minimize not just the strictly logical dependencies, but also the non-logical ones, like the sets of symbols and notations needed for a particular item, or particular automations like definitional expansions. For LCF-based proof assistants, this typically implies further work on the dependency tracking.

6 Evaluation, Experiments, and Applications

6.1 Dependency Extraction for CoRN and MML

We use the dependency extraction methods described in 3 and 4 to obtain fine dependency data for the CoRN library and an initial 100 article fragment of the MML. As described above, for CoRN, we use the dependency exporter implemented directly using the Coq code base. The export is thus approximately as fast as the Coq processing of CoRN itself, taking about 40 minutes on contemporary hardware. The product are for each CoRN file a corresponding file with dependencies, which have altogether about 65 MB. This information is then post-processed by scripts of off-the shelf tools into the dependency graph discussed below.

For Mizar and MML we use the brute-force dependency extraction approach discussed above. This takes significantly longer than Mizar processing alone; a number of preprocessing and normalization steps that need to be done when splitting articles into micro-articles also decreases performance. For our data this now takes about one day for the initial 100 article fragment of the MML. The main share of this time being spent on minimizing the large numbers of items used implicitly by Mizar. Note that in this implementation we are initially more interested in achieving completeness and minimality rather than efficiency, and a number of available optimizations can reduce this time significantly.¹²

In order to compare the benefits of having fine dependencies, we also compute for each library the *full file-based dependency* graph for all items. These graphs emulate the current dumb file-based treatment of dependencies in these libraries: each time an item is changed in some file, all items in the depending files have to be re-verified. The two kinds of graphs for both libraries are then compared in Table 1.

The graphs confirm our initial intuition that having the fine dependencies will significantly speed up partial recompilation of the large libraries, which is especially interesting in the CoRN and MML formal wikis that we develop.¹³ For example, the average number of items that need to be recompiled when a

¹² See Section 6.4.

¹³ <http://mws.cs.ru.nl/mwiki/>, <http://mws.cs.ru.nl/cwiki/>

random item is changed has dropped about seven times for **CoRN**, and about five times for **Mizar**. The medians for these numbers are even more interesting, increasing to fifteen for **Mizar**. The difference between **MML** and **CoRN** is also quite interesting, but it is hard to draw any conclusions. The corpora differ in their content and use different styles and techniques.

Table 1. Statistics of the item-based and file-based dependencies for **CoRN** and **MML**

	CoRN/item	CoRN/file	MML-100/item	MML-100/file
Items	9 462	9 462	9 553	9 553
Deps	175 407	2 214 396	704 513	21 082 287
TDeps	3 614 445	24 385 358	7 258 546	34 974 804
P(%)	8	54.5	15.9	76.7
ARL	382	2 577.2	759.8	3 661.1
MRL	12.5	1 183	155.5	2 377.5

Deps Number of dependency edges

TDeps Number of transitive dependency edges

P Probability that given two randomly chosen items, one depends (directly or indirectly) on the other, or vice versa.

ARL Average number of items recompiled if one item is changed.

MRL Median number of items recompiled if one item is changed.

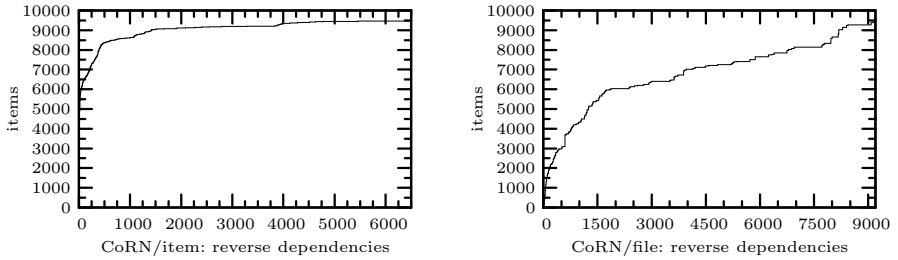


Fig. 1. Cumulative transitive reverse dependencies for **CoRN**: file-based vs. item-based

Table 2 shows statistics about the number and structure of *explicit* and *implicit* dependencies that we have done for **Mizar**. Explicit dependencies are anything that is already mentioned in the original text. Implicit dependencies are everything else, for example dependencies on type mechanisms (registrations, see also Section 6.4). Note that the ratio of implicit dependencies is very significant, which suggests that handling them precisely is essential for the learning and ATP experiments conducted in the next section.

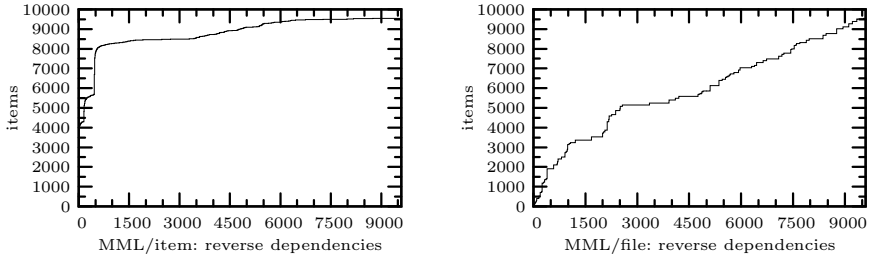


Fig. 2. Cumulative transitive reverse dependencies for MML: file-based vs. item-based

Table 2. Statistics of Mizar direct dependencies from and to different items

	theorem	top-level lemma	definition	scheme	registration
from	550 134	44 120	44 216	7 053	58 622
to	314 487	2 384	263 486	6 510	108 449

6.2 Dependency Analysis for AI-Based Proof Assistance

The knowledge of how a large number of theorems are proved is used by mathematicians to direct their new proof attempts and theory developments. In the same way, the precise formal proof knowledge that we now have can be used for directing formal automated theorem proving (ATP) systems and meta-systems over the large mathematical libraries. In [2] we provide an initial evaluation of the usefulness of our MML dependency data for machine learning of such proof guidance of first-order ATPs.

These experiments are conducted on a set of 2078 problems extracted from the Mizar library and translated to first-order ATP format. We emulate the growth of the Mizar library (limited to these 2078 problems) by considering all previous theorems and definitions when a new conjecture is attempted (i.e., when a new theorem is formulated by an author, requiring a proof). The ATP problems thus become very large, often containing thousands of the previously proved formulas as available axioms, which obviously makes automated theorem proving quite difficult [16,12]. We run the state-of-the-art **Vampire-SInE** [9] ATP system on these large problems, and solve 548 of them (with a 10-second timelimit). Then, instead of attacking such large problems directly, we use machine learning to learn proof relevance from all previous fine-grained proof dependencies.

This technique works surprisingly well: in comparison with running **Vampire-SInE** directly on the large problems, the problems pruned by trained machine learners can be proved by **Vampire** in 788 resp. 824 cases, depending on the kind of machine learning method (naive Bayes, resp. kernel-based) applied. This means that the efficiency of the automated theorem proving is raised by 43.7% resp. 50.4% when we use the knowledge about previous proof dependencies. This is a very significant improvement in the world of automated theorem proving, where the search complexity is typically superexponential.

In [3] we further leverage this automated reasoning technique by scaling the dependency analysis to the whole MML, and attempting a fully automated proof for every MML theorem. This yields the so-far largest number of fully automated proofs over the whole MML, allowing us (using the precise formal dependencies of the ATP and MML proofs) to attempt an initial comparison of human and automated proofs in general mathematics.

6.3 Interactive Editing with Fine-Grained Dependencies

A particular practical use of fine dependencies (initially motivating the work done on Coq dependencies in 3) is for advanced interactive editing. **tmEgg** [10] is a $\text{\TeX}_{\text{MACS}}$ -based user interface to Coq.¹⁴ Its main purpose is to integrate formal mathematics done in Coq in a more general document (such as course notes or journal article) without forcing the document to follow the structure of the formal mathematics contained therein.

For example, it does not require that the order in which the mathematical constructs appear in the document be the same as the order in which they are presented to Coq. As one would expect, the latter must respect the constraints inherent to the incremental construction of the formal mathematics, such as a lemma being proven before it is used in the proof of a theorem or a definition being made before the defined construct is used.

However, the presentation the author would like to put in the document may not strictly respect these constraints. For example, clarity of exposition may benefit from first presenting the proof of the main theorem, making it clear how each lemma being used is useful, and then only go through all lemmas. Or a didactic presentation of a subject may first want to go through some examples before presenting the full definitions for the concepts being manipulated.

tmEgg thus allows the mathematical constructs to be in any order in the document, and uses the dependency information to dynamically — and lazily — load any construct necessary to perform the requested action. For example, if the requested action is “check the proof of this theorem”, it will automatically load all definitions and lemmas used by the statement or proof of the theorem.

An interactive editor presents slightly different requirements than the batch recompilation scenario of a mathematical library described in Section 6.1. One difference is that an interactive editor needs dependency information, as part of the interactive session, for partial in-progress proofs. Indeed, if any in-progress proof depends on an item T , and the user wishes to change or unload (remove from the environment) T , then the part of the in-progress proof that depends on T has to be undone, even if the dependency is opaque.

¹⁴ The dependency tracking for Coq was actually started by the second author as part of the development of **tmEgg**. This facility has been already integrated in the official release of Coq. Since then this facility was extended to be able to treat the whole of the CoRN library. These changes are not yet included in the official release of Coq.

6.4 Learning Dependencies

In Section 6.2 it was shown that knowing the exact dependencies of previous proofs and learning from them can very significantly improve the chance of proving the next theorem fully automatically. Such AI techniques are however not limited just to fully automated proving, but can be used to improve the performance of many other tasks. One of them is also the actual process of obtaining exact minimal dependencies, as done for *Mizar* in Section 4.

This may seem impossible (or cyclic) at first: how can the process of determining minimal dependencies be practically improved by having minimal dependencies? The answer is that the (already computed) dependencies of the *previous* items can help significantly to guess the dependencies of the *next* item, and thus speed up the computation.

More precisely, for each *Mizar* item, we use its symbols (constructors) and explicit theorem references as the characterization (input features) for the learning. The output features (labels) are the registrations (implicit type mechanisms) needed for the item. For each item, we train the learner on all previous examples, and produce a prediction (registrations re-ordered by their predicted relevance) for the current item. The minimization algorithm is then modified to first do a binary search of this re-ordered list for its minimal initial segment sufficient for verifying the item, and then minimize this segment using the standard method. This is compared to the standard method on a random sample of 1000 theorems.

Tables 3 and 4 present our findings. The recommendation-assisted minimization algorithm was more than two times faster than unassisted minimization. Table 4 shows the speedup factors as the recommendation-assisted algorithm was applied to articles that are “deeper” in the MML, i.e., farther away from the set-theoretic axioms of *Mizar*. Our data confirms our intuition that, as the MML develops from the axioms of set theory to advanced mathematics, through smart recommendation we can considerably speed up dependency calculation.

Table 3. Summary of recommendation-assisted registration minimization

	Minimization time (sec)	Avg. minimization time/theorem (sec)
Unassisted	5755	5.76
Assisted	2433	2.43
Speedup Factor	2.37	
Number of cases		
Faster theorems	826	
Slower theorems	174	

7 Related Work

Related work exists in the first-order ATP field, where a number of systems can today output the axioms needed for a particular proof. Purely semantic

Table 4. Speedup through the library

Number of theorems	Unassisted time (sec)	Assisted time (sec)	Speedup Factor
200	143.81	90.18	1.59
400	967.90	541.70	1.79
600	1525.20	890.29	1.71
800	2828.82	1436.05	1.97
1000	5755.43	2433.26	2.37

(proof object) dependencies have been extracted several times for several ITPs, for example by Bertot and the Helm project for *Coq* [5,13,4], and Obua and McLaughlin for *HOL Light* and *Isabelle*. The focus of the latter two dependency extractions is on cross-verification, and are based on quite low-level (proof object) mechanisms. A higher-level¹⁵ semantic dependency exporter for *HOL Light* was recently implemented by Adams [1] for his work on *HOL Light* re-verification in *HOL Zero*. This could be usable as a basis for extending our applications to the core *HOL Light* library and the related large *Flyspeck* library. The *Coq/CoRN* approach quite likely easily scales to other large *Coq* libraries, like for example the one developed in the Math Components project [6]. Our focus in this work is wider than the semantic-only efforts: We attempt to get the full information about all implicit mechanisms (including syntactic mechanisms), and we are interested in using the information for smart re-compilation, which requires to track much more than just the purely semantic or low-level information.

8 Conclusion and Future Work

In this paper we have tried to show the importance and attractiveness of formal dependencies. We have implemented and used two very different techniques to elicit fine-grained proof dependencies for two very different proof assistants and two very different large formal mathematical libraries. This provides enough confidence that our approaches will scale to other important libraries and assistants, and our techniques and the derived benefits will be usable in other contexts.

Mathematics is being increasingly encoded in a computer-understandable (formal) and in-principle-verifiable way. The results are increasingly large interdependent computer-understandable libraries of mathematical knowledge. (Collaborative) development and refactoring of such large libraries requires advanced computer support, providing fast computation and analysis of dependencies, and fast re-verification methods based on the dependency information. As such automated assistance tools reach greater and greater reasoning power, the cost/benefit ratio of doing formal mathematics decreases.

Given our work on various parts of this program, providing exact dependency analysis and linking it to the other important tools seems to be a straightforward

¹⁵ By *higher-level* we mean tracking *higher-level* constructs, like use of theorems and tactics, not just tracking of the low-level primitive steps done in the proof-assistant's kernel.

choice. Even though the links to proof automation, fast large-scale refactoring, proof analysis, etc., are fresh, we believe that the significant performance boosts we've seen already sufficiently demonstrate the importance of good formal dependency analysis for formal mathematics, and for future mathematics in general.

References

1. Adams, M.: Introducing HOL Zero. In: Fukuda, K., Hoeven, J., Joswig, M., Takayama, N. (eds.) ICMS 2010. LNCS, vol. 6327, pp. 142–143. Springer, Heidelberg (2010)
2. Alama, J., Heskes, T., Kühlwein, D., Tsvitsivadze, E., Urban, J.: Premise selection for mathematics by corpus analysis and kernel methods. CoRR abs/1108.3446 (2012), <http://arxiv.org/abs/1108.3446>
3. Alama, J., Kühlwein, D., Urban, J.: Automated and Human Proofs in General Mathematics: An Initial Comparison. In: Bjørner, N., Voronkov, A. (eds.) LPAR-18 2012. LNCS, vol. 7180, pp. 37–45. Springer, Heidelberg (2012)
4. Asperti, A., Padovani, L., Coen, C.S., Guidi, F., Schena, I.: Mathematical knowledge management in HELM. Ann. Math. Artif. Intell. 38(1-3), 27–46 (2003)
5. Bertot, Y., Pons, O., Pottier, L.: Dependency graphs for interactive theorem provers. Tech. rep., INRIA, report RR-4052 (2000)
6. Garillot, F., Gonthier, G., Mahboubi, A., Rideau, L.: Packaging Mathematical Structures. In: Berghofer, S., Nipkow, T., Urban, C., Wenzel, M. (eds.) TPHOLs 2009. LNCS, vol. 5674, pp. 327–342. Springer, Heidelberg (2009)
7. Grabowski, A., Kornilowicz, A., Naumowicz, A.: mizar in a nutshell. Journal of Formalized Reasoning 3(2), 153–245 (2010)
8. Haftmann, F., Wenzel, M.: Constructive Type Classes in Isabelle. In: Altenkirch, T., McBride, C. (eds.) TYPES 2006. LNCS, vol. 4502, pp. 160–174. Springer, Heidelberg (2007)
9. Hoder, K., Voronkov, A.: Sine Qua Non for Large Theory Reasoning. In: Bjørner, N., Sofronie-Stokkermans, V. (eds.) CADE 2011. LNCS, vol. 6803, pp. 299–314. Springer, Heidelberg (2011)
10. Mamane, L., Geuvers, H.: A document-oriented Coq plugin for TeXmacs. In: Kauers, M., Kerber, M., Miner, R., Windsteiger, W. (eds.) MKM 2007 - Work in Progress. RISC Report, vol. 07-06, pp. 47–60. University of Linz, Austria (2007)
11. Matuszewski, R., Rudnicki, P.: Mizar: the first 30 years. Mechanized Mathematics and Its Applications 4, 3–24 (2005)
12. Meng, J., Paulson, L.C.: Lightweight relevance filtering for machine-generated resolution problems. J. Applied Logic 7(1), 41–57 (2009)
13. Pons, O., Bertot, Y., Rideau, L.: Notions of dependency in proof assistants. In: UITP 1998. Eindhoven University of Technology (1998)
14. Spitters, B., van der Weegen, E.: Type classes for mathematics in type theory. Mathematical Structures in Computer Science 21(4), 795–825 (2011)
15. Urban, J.: MoMM—fast interreduction and retrieval in large libraries of formalized mathematics. International Journal on Artificial Intelligence Tools 15(1), 109–130 (2006)
16. Urban, J., Hoder, K., Voronkov, A.: Evaluation of Automated Theorem Proving on the Mizar Mathematical Library. In: Fukuda, K., van der Hoeven, J., Joswig, M., Takayama, N. (eds.) ICMS 2010. LNCS, vol. 6327, pp. 155–166. Springer, Heidelberg (2010)
17. Wiedijk, F.: Mizar's Soft Type System. In: Schneider, K., Brandt, J. (eds.) TPHOLs 2007. LNCS, vol. 4732, pp. 383–399. Springer, Heidelberg (2007)